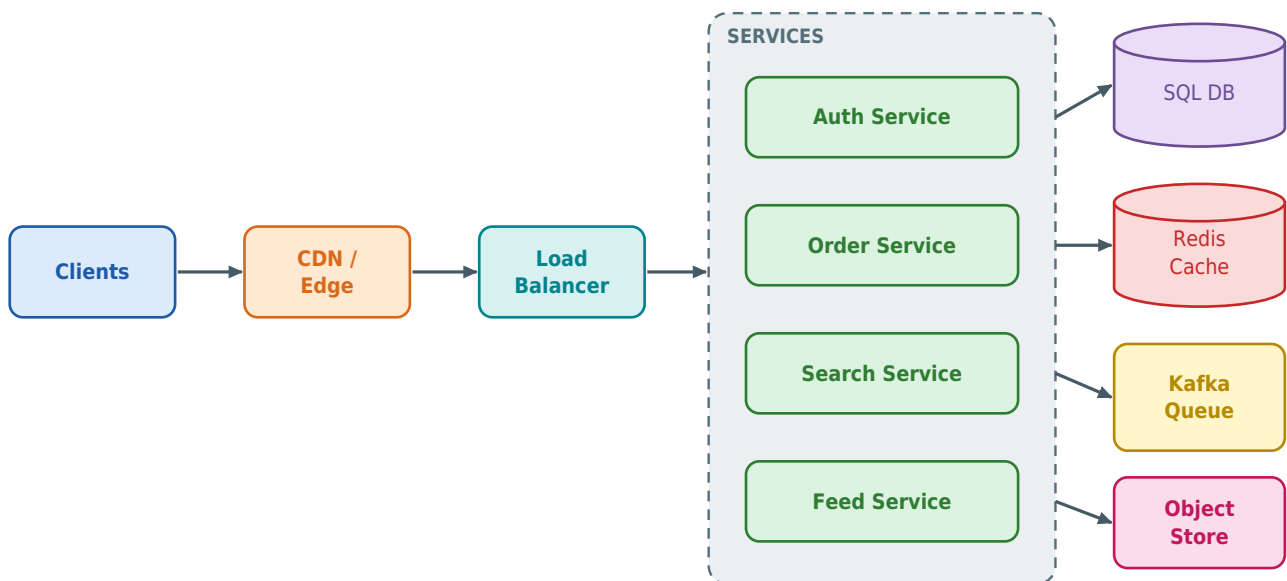


Mastering System Design

High-Level Design (HLD)

A complete, diagram-first guide: from networking fundamentals and estimation to databases, messaging, distributed systems, reliability and real-world architecture.



INSIDE THIS BOOK

Foundations

APIs

Databases

Scaling

Messaging

Distributed

Reliability

Security

Storage

Table of Contents

PART I — SYSTEM DESIGN FOUNDATIONS	7
1. Introduction to System Design	8
1.1 What Is System Design?	8
1.2 HLD, LLD and Software Architecture	9
1.3 Building Blocks of a Distributed Application	9
1.4 System Styles and Deployment Shapes	10
1.5 Requirements: Functional and Non-Functional	12
1.6 Quality Attributes	12
1.7 Core Trade-offs	14
1.8 Bottlenecks and Single Points of Failure	15
1.9 Request Lifecycle	15
1.10 Interview Workflow and Communication	16
2. Requirements Engineering	18
2.1 Understanding the Problem	18
2.2 Actors and Use Cases	19
2.3 Traffic, Users and Growth	19
2.4 Data Requirements	20
2.5 Performance and Reliability Requirements	20
2.6 Security, Compliance and Regional Requirements	21
2.7 Disaster Recovery, SLA, SLO and SLI	21
2.8 API Requirements and Dependencies	22
2.9 Hot Spots and Assumptions	23
3. Back-of-the-Envelope Estimation	24
3.1 Why Estimation Matters	24
3.2 Numbers Every Engineer Should Know	24
3.3 From Users to Traffic	26
3.4 Request Size and Bandwidth	26
3.5 Storage Estimation	27
3.6 Cache and Memory Estimation	27
3.7 Servers, Replicas and Shards	27
3.8 Messaging, Connections and CDN	28
3.9 Peak, Growth and Safety Margins	28
3.10 Worked Examples at 1K, 1M, 100M and 1B Users	29
4. Networking Fundamentals	31
4.1 What Happens When You Enter a URL	31
4.2 Network Models: OSI and TCP/IP	32
4.3 IP Addressing, Ports and Sockets	32
4.4 TCP: Reliable Transport	33

4.5 UDP and QUIC	34
4.6 DNS: The Internet's Phone Book	35
4.7 HTTP and HTTPS	36
4.8 TLS/SSL, Certificates	37
4.9 Connection Management and Latency	38
4.10 NAT, Firewalls and Proxies	38
4.11 L4 vs L7 and WebSockets	39
4.12 Failures: Partitions and Timeouts	40

PART II — APIS AND APPLICATION ARCHITECTURE 41

5. API Design 42

5.1 API Fundamentals and REST	42
5.2 HTTP Methods and Semantics	43
5.3 HTTP Status Codes, Headers and Bodies	44
5.4 REST vs RPC, gRPC and GraphQL	45
5.5 Listing Data: Pagination, Filtering, Sorting, Search	47
5.6 Evolving APIs: Versioning and Compatibility	47
5.7 Protecting and Hardening an API	48
5.8 Documenting, Composing and Asynchronous APIs	49

6. Monoliths, Modular Monoliths and Microservices 51

6.1 Monolithic Architecture	51
6.2 Modular Monolith	52
6.3 Microservice Architecture	52
6.4 The Price of Distribution	54
6.5 Migration and Alternatives	54
6.6 Choosing the Right Architecture	56

7. API Gateway, Reverse Proxy and Service Discovery 57

7.1 Reverse Proxy	57
7.2 API Gateway	57
7.3 Service Discovery	60
7.4 Traffic Direction and Service Mesh	61

PART III — DATA AND DATABASE ARCHITECTURE 63

8. Database Fundamentals 64

8.1 What a Database Is	64
8.2 The Relational Model	65
8.3 Transactions and ACID	66
8.4 Connections, Planning and Locking	67
8.5 Workload Types and Storage Layouts	68

9. SQL vs NoSQL 70

9.1 Relational Databases	70
9.2 NoSQL Databases	71

1.2 HLD, LLD and Software Architecture

High-Level Design vs Low-Level Design

Aspect	High-Level Design (HLD)	Low-Level Design (LLD)
Focus	Components, services, data flow, infrastructure	Classes, interfaces, methods, algorithms
Question answered	"What are the big boxes and how do they connect?"	"How is each box built internally?"
Artifacts	Architecture diagram, API list, data model, capacity plan	Class diagram, sequence diagram, design patterns, code
Concerns	Scale, availability, consistency, latency, cost	Readability, SOLID, extensibility, testability
Typical interview	"Design Twitter / WhatsApp / URL shortener"	"Design a parking lot / ATM / elevator"

System Design vs Software Architecture

Software architecture is the set of structural decisions that are expensive to change — layers, module boundaries, communication styles, technology families. System design is the act of producing and evaluating such a structure for a given problem, including capacity and failure analysis. In practice the words overlap: architects do system design, and system-design work produces an architecture.

REAL-WORLD NOTE

In day-to-day work, HLD is written in a design document (RFC/ADR) reviewed by peers before implementation. LLD lives later, inside each service's code and its own design notes.

1.3 Building Blocks of a Distributed Application

Components of a distributed application

A distributed application is a collection of independent processes on different machines that cooperate over a network and appear as one system to the user. Almost every large system is assembled from the same small set of blocks.

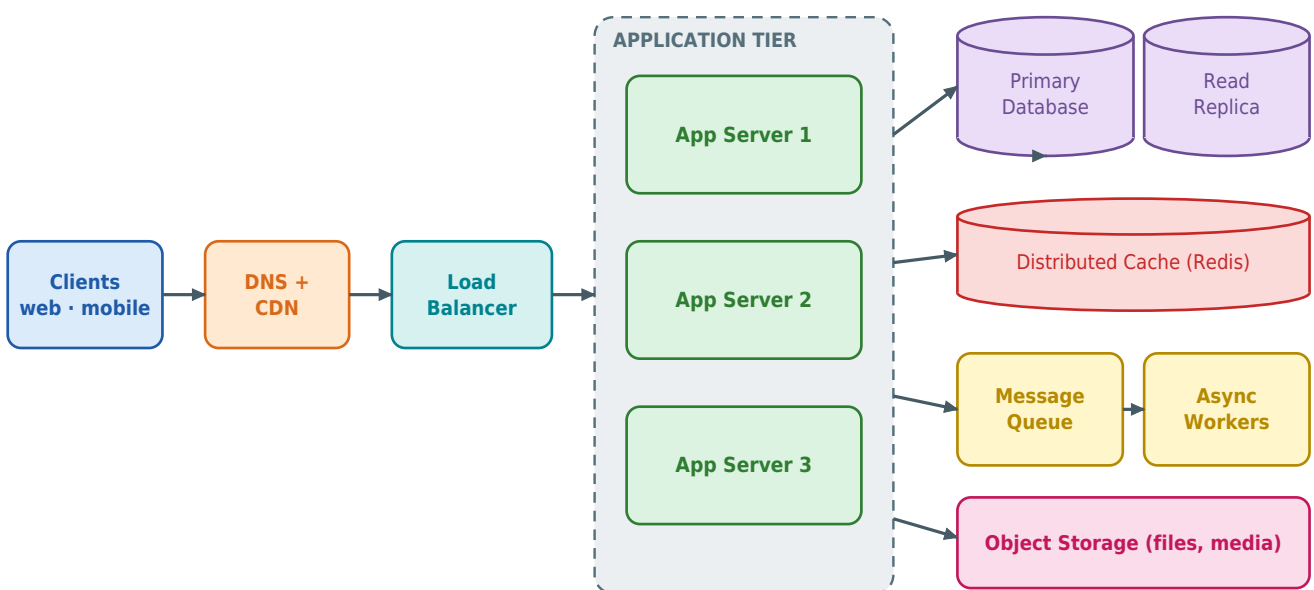


Figure 1.1 — Anatomy of a typical distributed web application

3.10 Worked Examples at 1K, 1M, 100M and 1B Users

Using the same behaviour model (50 reads and 2 writes per DAU per day, 500 B per post, peak factor 3):

Scale (DAU)	Reads/day	Avg read QPS	Peak read QPS	Avg write QPS	Storage/year (raw)
1 K	50 K	0.6	~2	0.02	~0.4 GB
1 M	50 M	580	~1.7 K	23	~365 GB
100 M	5 B	58 K	~174 K	2.3 K	~36.5 TB
1 B	50 B	580 K	~1.7 M	23 K	~365 TB

Complexity and cost rise with each order of magnitude

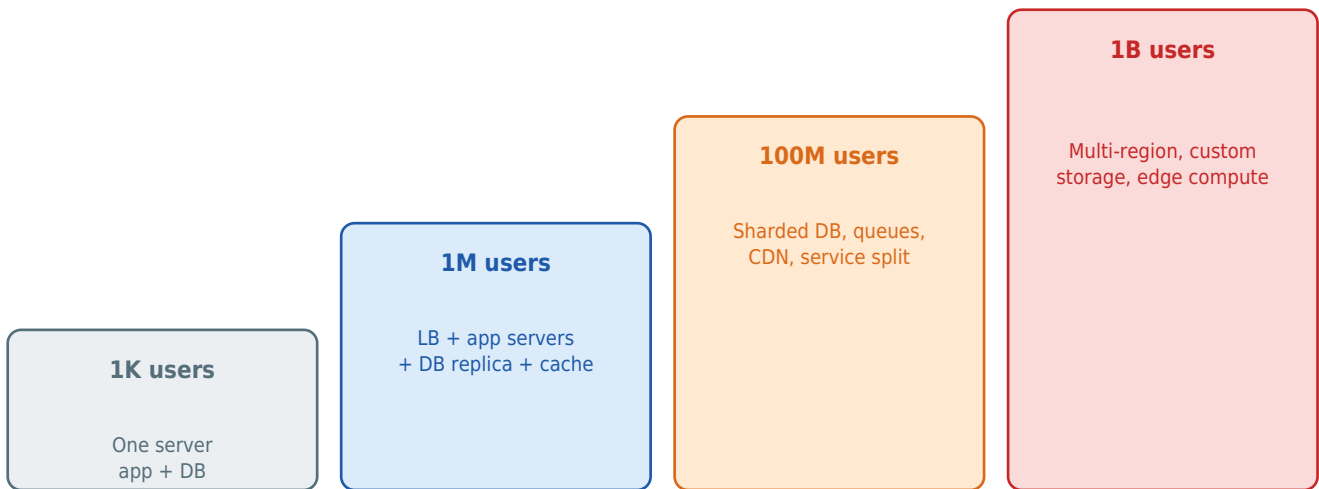


Figure 3.4 — Typical architecture stage at each user scale

1,000 users

A single small server with the application and database is enough. Add backups and monitoring; spend effort on features, not scale.

1,000,000 users

Peak ~1.7 K read QPS: one strong DB can handle it, but add a load balancer with 2–3 stateless app servers, a Redis cache, a read replica and a CDN for static assets for availability.

100,000,000 users

Peak ~174 K read QPS and 143 TB/year: shard the database, cache heavily, use queues for fan-out, split into services and put media on object storage behind a CDN.

CHAPTER

16

Caching and Redis

Cache strategies, invalidation, eviction, stampede, and Redis structures, persistence and clustering

IN THIS CHAPTER

16.1 Why Cache?

16.2 Where Caches Live

16.3 Caching Strategies

16.4 Expiry, Invalidation and Consistency

16.5 Eviction Policies

16.6 Warming and Failure Patterns

16.7 Redis

16.8 Redis Durability, Replication and Scaling

16.1 Why Cache?

Why caching improves performance

A **cache** stores copies of frequently used or expensive-to-compute data in faster storage so later requests are served without repeating the work. Memory is about 100–1000× faster than disk-based databases and avoids network round trips and computation. Caching typically delivers three wins at once: **lower latency**, **higher throughput** and **lower backend load and cost**.

Cache hit and cache miss

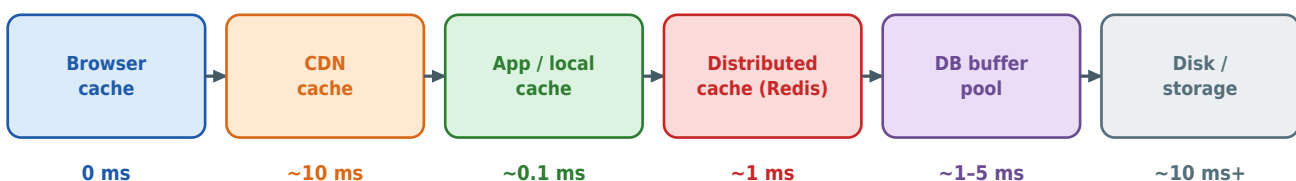
A **hit** means the data is found in the cache and returned immediately; a **miss** means it is not there, so the system fetches it from the source (slow path) and usually stores it.

Cache hit ratio

$\text{hit_ratio} = \text{hits} \div (\text{hits} + \text{misses})$. At 90% hit ratio only 10% of reads reach the database; raising it from 90% to 99% cuts database load by another 10×. Average latency = $\text{hit_ratio} \times \text{cache_latency} + (1 - \text{hit_ratio}) \times \text{db_latency}$, so misses dominate.

16.2 Where Caches Live

A request travels right until some layer has the answer (a HIT stops the journey)



The closer to the user, the faster — and the harder to invalidate

Figure 16.1 — Caching happens at every layer between the user and the disk

CHAPTER

19

Kafka Deep Dive

Brokers, partitions, consumer groups, replication and ISR, transactions, compaction and operations

IN THIS CHAPTER

19.1 Kafka Architecture

19.2 Producing and Consuming

19.3 Replication and Durability

19.4 Offsets and Delivery Semantics

19.5 Retention, Compaction and Schemas

19.6 Operating Kafka

19.1 Kafka Architecture

Kafka architecture

Apache Kafka is a distributed, partitioned, replicated **commit log**. Producers append records to topics; consumers read them at their own pace by tracking offsets. Unlike a classic queue, records are **not deleted when read**; they are retained by policy, so many independent consumers can read, re-read and replay the same data. This makes Kafka both a messaging system and the backbone of event streaming.

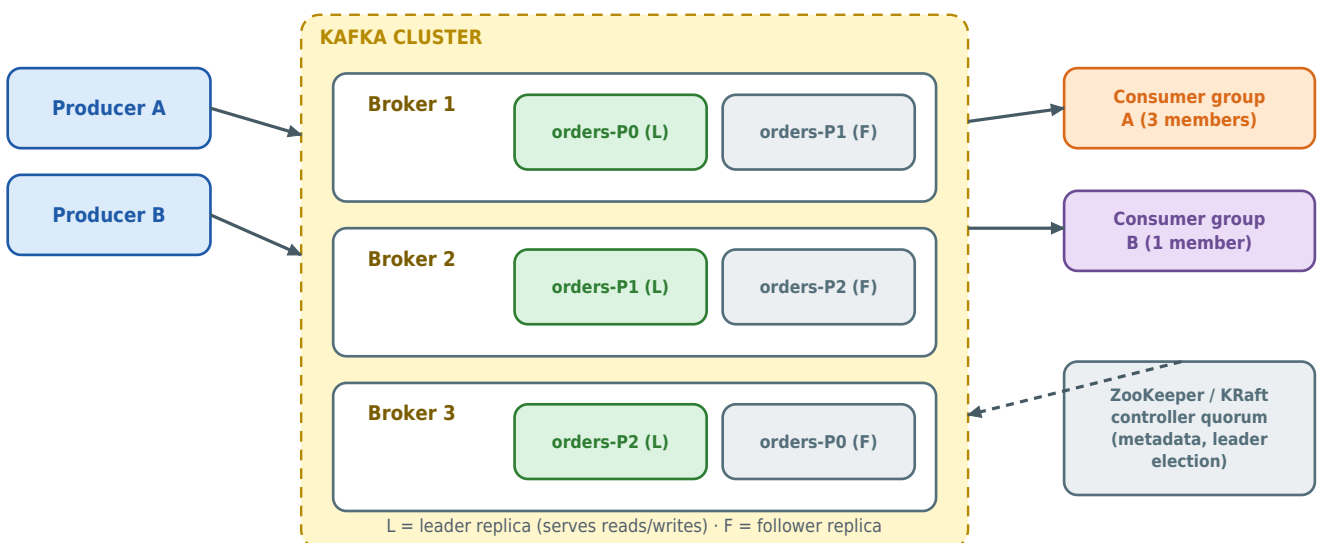


Figure 19.1 — Kafka architecture: brokers, partitioned topics with replicas, producers and consumer groups

Kafka cluster

A group of broker servers working together (typically 3 to hundreds). Cluster metadata (topics, partitions, leaders) is managed by a controller quorum — ZooKeeper in older versions, **KRaft** (Kafka's built-in Raft) in current ones.

Brokers

A broker is one Kafka server storing partitions on disk and serving producers and consumers. Any broker can answer metadata requests; each partition has exactly one **leader** broker at a time.

Topics

A named stream of records, split into partitions (**orders**, **payments**, **clicks**).

CHAPTER

22

CAP and Consistency

CAP, PACELC, consistency models, quorums, vector clocks, CRDTs and choosing consistency per feature

IN THIS CHAPTER

22.1 The CAP Theorem

22.2 Consistency Models

22.3 Quorums and Conflicts

22.4 Beyond CAP

22.1 The CAP Theorem

CAP theorem

CAP (Brewer, proved by Gilbert and Lynch) says that a distributed data store cannot simultaneously guarantee all three of **Consistency**, **Availability** and **Partition tolerance**. More precisely: when a network partition occurs, you must choose between consistency and availability.

During a network **PARTITION** you must choose: **Consistency** or **Availability**

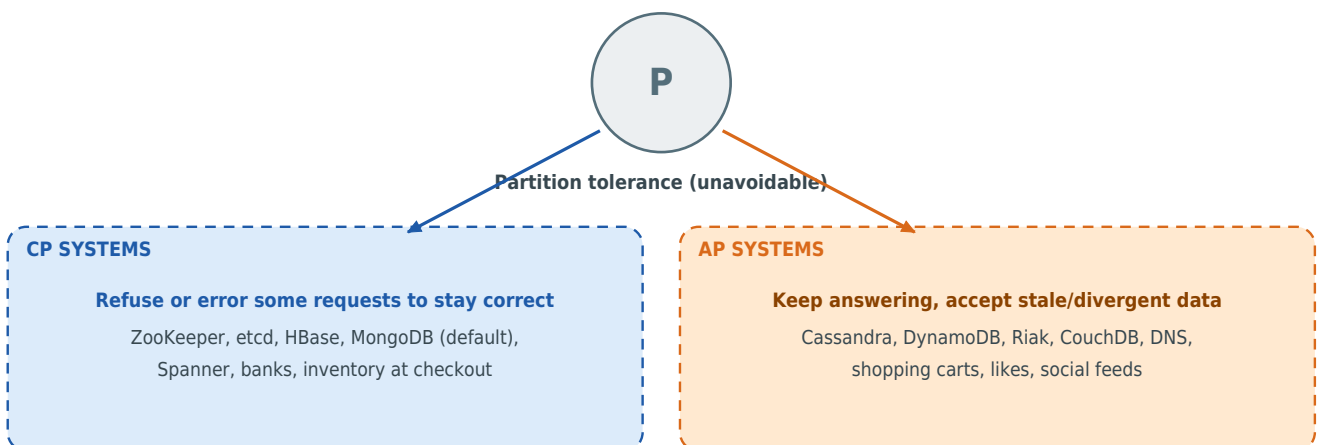


Figure 22.1 — CAP: when a partition happens, choose consistency (CP) or availability (AP)

Consistency

In CAP, consistency means **linearizability**: every read returns the most recent completed write, as if there were a single copy of the data. (This is not the "C" of ACID, which means constraints are preserved.)

Availability

Every request to a non-failing node receives a (non-error) response, without guarantee it contains the latest write.

Partition tolerance

The system continues operating despite messages being lost or delayed between nodes.